

Optativa 1.2: Máquina virtual con pool logical (LVM)

Objetivo

En esta práctica voy a crear una máquina virtual cuyo disco principal no sea un fichero de imagen almacenado en un directorio, sino un **volumen lógico LVM**. Para conseguirlo crearé un grupo de volúmenes dedicado, definiré sobre él un pool de almacenamiento libvirt de tipo `logical`, crearé un volumen lógico de al menos 5 GB e instalaré una máquina virtual Linux utilizando ese volumen como disco `vda`.

Importante: no se debe utilizar el grupo de volúmenes del sistema operativo. Es necesario crear o reutilizar un grupo de volúmenes distinto y destinado a la práctica.

Resultado esperado

Al terminar tendré esta estructura:

```
Host
├── vg_libvirt_logical          Grupo de volúmenes LVM
independiente
├── disco-mv-lvm              Volumen lógico de 5 GB
│   ├── /dev/vg_libvirt_logical/disco-mv-lvm
│   └── mv-lvm-sergio        Máquina virtual que usa el
volumen como vda

libvirt
└── pool_logical_sergio      Pool de tipo logical sobre
vg_libvirt_logical
```

Requisitos previos

Necesito tener instaladas las herramientas de virtualización y LVM en el host:

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients virtinst lvm2
```

Compruebo que el servicio de libvirt está disponible:

```
sudo systemctl status libvirtd
```

En distribuciones actuales puede usarse `virtqemud` en lugar de `libvirtd` :

```
sudo systemctl status virtqemud
```

También necesito un disco completo o una partición libre que no contenga datos importantes. En los ejemplos se utiliza `/dev/sdb` , pero cada persona debe sustituirlo por el dispositivo libre de su propio host.

Advertencia: los comandos `pvcreate` , `vgcreate` y `wipefs` modifican o destruyen datos del dispositivo indicado. Antes de ejecutarlos hay que verificar cuidadosamente el nombre del disco o partición.

1. Identificar espacio para LVM

Primero compruebo los discos, particiones y configuraciones LVM existentes:

```
lsblk -f  
sudo pvs  
sudo vgs  
sudo lvs
```

`lsblk -f` muestra los discos, las particiones, sistemas de ficheros y puntos de montaje. Los comandos `pvs` , `vgs` y `lvs` muestran, respectivamente, los volúmenes físicos, grupos de volúmenes y volúmenes lógicos ya existentes.

No debo usar el VG del sistema, que en mi caso podría llamarse `vg_debian` . Necesito localizar un disco o partición sin utilizar. Si dispongo de un disco vacío, por ejemplo `/dev/sdb` , puedo comprobar que no tiene particiones montadas con:

```
lsblk /dev/sdb  
sudo wipefs -n /dev/sdb
```

La opción `-n` solo muestra firmas existentes y no borra nada.

Alternativa: no tengo disco libre

Si no dispongo de ningún disco o partición libre en el host, puedo realizar la práctica dentro de una máquina virtual con virtualización anidada. Esa máquina debe tener un disco adicional para

LVM y acceso a las extensiones de virtualización del procesador; al crearla se puede usar, por ejemplo:

```
--cpu host-passthrough
```

Dentro de esa máquina instalaría `qemu-kvm`, `libvirt`, `virt-install` y `lvm2`, y seguiría el resto de los pasos. En esta guía continúo con el caso habitual: crear el pool directamente en el host.

2. Crear el grupo de volúmenes dedicado

2.1. Crear el volumen físico

Inicializo el disco o partición libre como volumen físico LVM. En este ejemplo el dispositivo libre es `/dev/sdb`:

```
sudo pvcreate /dev/sdb
```

Compruebo que se ha creado correctamente:

```
sudo pvs
```

La salida debe mostrar `/dev/sdb` como un PV, aunque todavía no esté asignado a ningún grupo de volúmenes.

2.2. Crear el grupo de volúmenes

Creo un grupo de volúmenes exclusivo para el pool de libvirt:

```
sudo vgcreate vg_libvirt_logical /dev/sdb
```

Compruebo el grupo creado y su espacio disponible:

```
sudo vgs  
sudo vgdisplay vg_libvirt_logical
```

La columna `vFree` de `vgs` indica el espacio que queda libre en el grupo. Para crear el volumen de la práctica necesito al menos 5 GB libres; es recomendable disponer de algo más de espacio.

El resultado esperado es similar a este:

VG	#PV	#LV	#SN	Attr	VSize	VFree
vg_libvirt_logical	1	0	0	wz--n-	<10.00g	<10.00g

Con esto queda separado el almacenamiento de la práctica del almacenamiento del sistema operativo.

3. Definir el pool logical de libvirt

Un pool `logical` de libvirt utiliza un grupo de volúmenes LVM como origen y trata los volúmenes lógicos como discos de las máquinas virtuales.

Creo el fichero `/tmp/pool-logical-sergio.xml`:

```
cat > /tmp/pool-logical-sergio.xml <<'EOF'
<pool type='logical'>
  <name>pool_logical_sergio</name>
  <source>
    <name>vg_libvirt_logical</name>
    <format type='lvm2' />
  </source>
  <target>
    <path>/dev/vg_libvirt_logical</path>
  </target>
</pool>
EOF
```

Muestro el contenido para revisarlo antes de registrarlo en libvirt:

```
cat /tmp/pool-logical-sergio.xml
```

Defino el pool, lo inicio y activo el arranque automático:

```
sudo virsh pool-define /tmp/pool-logical-sergio.xml
sudo virsh pool-start pool_logical_sergio
sudo virsh pool-autostart pool_logical_sergio
```

Compruebo todos los pools disponibles:

```
sudo virsh pool-list --all
```

La salida debe incluir una línea similar a esta:

Nombre	Estado	Inicio automático

pool_logical_sergio	activo	si

Finalmente, compruebo la definición XML que almacena libvirt:

```
sudo virsh pool-dumpxml pool_logical_sergio
```

Una salida válida tiene esta estructura:

```
<pool type='logical'>
  <name>pool_logical_sergio</name>
  <uuid>...</uuid>
  <capacity unit='bytes'>...</capacity>
  <allocation unit='bytes'>0</allocation>
  <available unit='bytes'>...</available>
  <source>
    <name>vg_libvirt_logical</name>
    <format type='lvm2' />
  </source>
  <target>
    <path>/dev/vg_libvirt_logical</path>
  </target>
</pool>
```

Los elementos importantes son:

- `type='logical'` : el pool se basa en LVM.
- `<name>vg_libvirt_logical</name>` dentro de `source` : indica el grupo de volúmenes que se usa.
- `/dev/vg_libvirt_logical` : es el directorio donde LVM expone los volúmenes lógicos como dispositivos de bloques.

4. Crear el volumen lógico de la máquina

Creo un volumen de 5 GB dentro del pool. Libvirt creará el volumen lógico correspondiente dentro de `vg_libvirt_logical` :

```
sudo virsh vol-create-as pool_logical_sergio disco-mv-lvm 5G --format raw
```

Compruebo que se ha creado:

```
sudo virsh vol-list pool_logical_sergio
```

La salida debe ser similar a:

Nombre	Ruta

disco-mv-lvm	/dev/vg_libvirt_logical/disco-mv-lvm

También puedo comprobarlo desde LVM directamente:

```
sudo lvs vg_libvirt_logical
```

El volumen lógico debe aparecer con el nombre `disco-mv-lvm` y un tamaño aproximado de 5 GB.

5. Instalar una MV sobre el volumen lógico

Para instalar la nueva máquina virtual necesito una ISO Linux. En este ejemplo se usa una ISO de Debian ubicada en `/var/lib/libvirt/isos/debian.iso` ; se debe sustituir por la ruta real de la ISO disponible en el host.

Creo la MV `mv-lvm-sergio` usando directamente el volumen LVM como disco principal:

```
sudo virt-install \
  --name mv-lvm-sergio \
  --memory 2048 \
  --vcpus 2 \
  --disk path=/dev/vg_libvirt_logical/disco-mv-lvm,format=raw,bus=virtio \
  --cdrom /var/lib/libvirt/isos/debian.iso \
  --network network=default,model=virtio \
  --os-variant debian12 \
  --graphics spice
```

Explicación de los parámetros relevantes:

- `--name mv-lvm-sergio` : nombre de la nueva máquina virtual.
- `--memory 2048` : asigna 2 GB de memoria RAM.
- `--vcpus 2` : asigna dos vCPU.
- `--disk path=/dev/vg_libvirt_logical/disco-mv-lvm,...` : usa el volumen lógico como disco de la MV.

- `format=raw` : un volumen lógico se entrega a QEMU como dispositivo de bloque sin formato `qcow2` .
- `bus=virtio` : utiliza un dispositivo de disco paravirtualizado.
- `--cdrom` : monta la ISO para realizar la instalación.
- `--network network=default` : conecta inicialmente la máquina a la red NAT por defecto de libvirt.

Completo la instalación del sistema operativo desde la consola gráfica o desde el visor de la máquina virtual. Tras finalizar, apago o expulso la ISO del CD-ROM si ya no es necesaria.

6. Comprobar el disco en el XML de la MV

Una vez creada la máquina, compruebo que su disco principal apunta al volumen lógico y no a un fichero de imagen:

```
sudo virsh dumpxml mv-lvm-sergio | grep -A12 -B2 'disco-mv-lvm'
```

El fragmento esperado debe contener algo equivalente a lo siguiente:

```
<disk type='block' device='disk'>
  <driver name='qemu' type='raw' cache='none' io='native' discard='unmap' />
  <source dev='/dev/vg_libvirt_logical/disco-mv-lvm' />
  <target dev='vda' bus='virtio' />
</disk>
```

La comprobación es correcta si se observan estas dos características:

1. El disco principal es de tipo bloque:

```
<disk type='block' device='disk'>
```

2. La fuente apunta al volumen lógico:

```
<source dev='/dev/vg_libvirt_logical/disco-mv-lvm' />
```

Es normal que también aparezca un dispositivo de tipo `file` para el CD-ROM. Ese dispositivo representa la ISO de instalación y no el disco principal de la máquina virtual.

7. Comprobaciones entregables

Para entregar la práctica ejecuto y guardo estas salidas:

```
sudo virsh pool-list --all
sudo virsh pool-dumpxml pool_logical_sergio
sudo virsh vol-list pool_logical_sergio
sudo virsh dumpxml mv-lvm-sergio | grep -A12 -B2 'disco-mv-lvm'
```

Estas comprobaciones demuestran, respectivamente:

- Que el pool logical está activo y configurado con autoinicio.
- Que el pool utiliza el VG correcto y es de tipo `logical`.
- Que existe un volumen lógico dentro del pool.
- Que el disco principal de la MV apunta directamente al volumen lógico.

8. Comparación: pool dir frente a pool logical

Pool `dir`

Un pool de tipo `dir` almacena los discos de las máquinas virtuales como ficheros dentro de un directorio del host, por ejemplo `/var/lib/libvirt/images/`.

Ejemplos de discos en un pool `dir`:

```
/var/lib/libvirt/images/debian.qcow2
/var/lib/libvirt/images/servidor.raw
```

Sus ventajas principales son:

- Es sencillo de entender y administrar: cada disco es un fichero.
- Se puede mover o copiar una máquina con herramientas habituales de ficheros, aunque siempre hay que tener en cuenta la definición XML y posibles discos adicionales.
- Permite utilizar el formato `qcow2`, que ofrece snapshots a nivel de imagen, compresión, cifrado y aprovisionamiento fino.
- Es habitual para laboratorios y escenarios pequeños.

Pool `logical`

Un pool de tipo `logical` usa un grupo de volúmenes LVM. Cada disco de una máquina virtual es un volumen lógico y aparece como un dispositivo de bloques, por ejemplo:

```
/dev/vg_libvirt_logical/disco-mv-lvm
```

Sus ventajas principales son:

- QEMU accede directamente a un dispositivo de bloques, sin utilizar un fichero de imagen sobre un sistema de archivos.
- LVM permite administrar el tamaño de forma flexible. Si el VG tiene espacio libre, se puede ampliar un volumen lógico con `lvextend`.
- La gestión de los discos queda integrada con las herramientas de LVM (`pvs` , `vgs` , `lvs` , `lvextend` , snapshots LVM, etc.).

Sin embargo, al utilizar un volumen lógico como disco raw se pierden características específicas de `qcow2` :

- No se usan snapshots propios de `qcow2`.
- No se dispone de compresión propia del formato.
- No se dispone de aprovisionamiento fino propio de `qcow2` a nivel de imagen.
- No se puede copiar la MV como un único fichero de disco; hay que copiar, clonar o exportar el volumen lógico.

LVM permite crear snapshots de volúmenes lógicos, pero son distintos de los snapshots de `qcow2` y requieren espacio libre adicional en el grupo de volúmenes. Por tanto, un pool `logical` puede resultar muy útil para trabajar a nivel de bloque y aprovechar la administración de LVM, mientras que un pool `dir` con `qcow2` suele ser más cómodo para movilidad, copias y snapshots de imagen.

Conclusión

He creado un grupo de volúmenes LVM independiente del sistema, he definido un pool libvirt de tipo `logical`, he creado dentro de él un volumen de 5 GB y he instalado una máquina virtual cuyo disco principal utiliza directamente ese volumen lógico. La verificación final se realiza comprobando el XML del pool y de la MV, donde debe aparecer el dispositivo `/dev/vg_libvirt_logical/disco-mv-lvm` como origen del disco principal de tipo `block`.